

交互模板

- C++ 代码:

```
#include <cstdio>
#include <iostream>

int main() {
    for (int l = 1, r = 1000000000, mid = (l + r) >> 1, res; l <= r; mid = (l + r) >> 1) {
        std::cout << mid << std::endl;
        std::cin >> res;
        if (res == 0) {
            return 0;
        } else if (res == -1) {
            l = mid + 1;
        } else if (res == 1) {
            r = mid - 1;
        } else {
            puts("OvO, I AK IOI"); // this statement will never be executed.
        }
    }
    return 0;
}
```

- 交互特点:

- $mid = (l + r) \gg 1$ 意味着mid是右移，相当于除以2
- res在这里是存储交互机答案
- `std::cout << mid << std::endl;` 记得用endl来清除缓存
- 或者`std::cout << std::flush`
- `std::cin >> res;` 读取输入流数据
- 一般来说输出和读取都在循环里面进行若干次
- 输出答案之后剩下的都差不多

- 改进思路:

- 一般交互题考虑二分或顺序输出

-

快读

```
#define rep(i,m,n) for(int i=m;i<=n;++i)

inline int read(){
    int s = 0, w = 1;
```

```

    char ch = getchar();
    while(ch < '0' || ch > '9'){if(ch == '-')w = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9') s = s * 10 + ch - '0',ch = getchar();
    return s * w;
}
const int MAXN = 20010;
int n;
int l[MAXN], r[MAXN], f[MAXN][2];

int main(){
    n = read();
    rep(i, 1, n) l[i] = read(), r[i] = read();

}

```

快写

```

void write(int x) {
    if (x < 0) putchar('-'), x = -x;
    if (x > 9) write(x / 10);
    putchar(x % 10 + '0');
}

```

qpow简洁版

```

using ll = long long;

ll qpow(ll a, ll b, ll mod) {
    ll res = 1;
    a %= mod;
    while (b > 0) {
        if (b & 1) res = (res * a) % mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return res;
}

```

快速乘

```

ll qmul(ll x, ll y, ll m) { // 快速模乘: 计算 (x * y) % m, 避免大数乘法溢出
    ll z = (long double) x / m * y + 0.5L; // 将 x / m * y 转为浮点数运算, 估算商 z,
    z ≈ (x * y) / m
    u64 c = (unsigned long long) x * y - (unsigned long long) z * m; // 计算 c = (x

```

```

* y) - z * m, 得到模 m 后的结果
    return c < m ? (ll) c : (ll) (c + m); // 如果 c < m, 直接返回 c; 否则返回 c + m,
    保证结果在 [0, m) 范围内
}

```

快速幂

```

// 快速幂: 计算 (a^n) % m, 高效处理大指数
ll qpow(ll a, ll n, ll m) {
    ll res = 1; // 初始化结果为 1
    while (n) { // 当 n > 0 时循环
        if (n & 1) res = qmul(res, a, m); // 如果 n 的最低位为 1, res = res * a % m
        a = qmul(a, a, m); // a = a * a % m
        n >>= 1; // n 右移一位
    }
    return res; // 返回 (a^n) % m
}

```

- 记住用法:
 - 题目要求 $(a^n) \% m$, 直接调用 `qpow(a, n, m)`。
 - 题目要求大数乘法 $(x * y) \% m$, 用 `qmul(x, y, m)`。

费马小定理求乘法逆元

- mod必须是质数

```

using ll = long long;

// 依赖之前的 qpow 函数
ll qpow(ll a, ll b, ll mod) {
    ll res = 1;
    a %= mod;
    while (b > 0) {
        if (b & 1) res = (res * a) % mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return res;
}

// 求 a 在 mod 下的逆元
ll inv(ll a, ll mod) {
    return qpow(a, mod - 2, mod);
}

```

除法取模

- $(a * \text{qpow}(b, p-2, p)) \% p$

```
// 假设要求 (A / B) % mod
// 这里的 mod 必须是质数

ll inv_B = qpow(B, mod - 2, mod); // 算出 B 的逆元
ll ans = (A * inv_B) % mod;      // 变成乘法运算
```

扩展欧几里得求逆元

- mod 不是质数, 但满足 $\text{gcd}(a, \text{mod}) = 1$ 。
- 原理: 求解 $ax + by = 1$, 则 x 即为逆元。

```
using ll = long long;

ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    ll d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}

ll inv_exgcd(ll a, ll mod) {
    ll x, y;
    ll d = exgcd(a, mod, x, y);
    if (d != 1) return -1; // 不存在逆元
    return (x % mod + mod) % mod; // 调整为正数
}
```

bool isPrime

```
bool is_prime(int x) {
    // 1. 处理小于2的特殊情况 (0和1不是素数)
    if (x < 2) return false;

    // 2. 核心循环: 只枚举到 sqrt(x)
    // 写法注意: 用 i <= x / i 哪怕 x 接近 INT_MAX 也不会溢出
    for (int i = 2; i <= x / i; i++) {
        if (x % i == 0) return false; // 只要发现一个因子, 就立即判定为合数
    }
}
```

```
// 3. 挺过循环没死，就是素数
return true;
}
```

素数埃氏筛 $O(n \log \log n)$

```
const int N = 1e6 + 10; // 根据题目范围调整
bool is_composite[N]; // 标记数组: false表示是素数, true表示是合数
// 也可以用 vector<int> primes 存储具体的素数

void get_primes(int n) {
    // 初始化: 0和1不是素数, 手动标记 (虽然算法里一般从2开始, 但为了严谨)
    is_composite[0] = is_composite[1] = true;

    // 核心循环
    for (int i = 2; i * i <= n; i++) { // 优化1: 只需要枚举到 sqrt(n)
        if (!is_composite[i]) {
            // 如果 i 没有被标记过, 说明 i 是素数
            // 接下来把 i 的所有倍数都标记为合数
            for (int j = i * i; j <= n; j += i) { // 优化2: 从 i*i 开始
                is_composite[j] = true;
            }
        }
    }
}

int main() {
    int n = 100;
    get_primes(n);

    for(int i = 2; i <= n; i++) {
        if (!is_composite[i]) cout << i << " ";
    }
    return 0;
}
```

欧拉筛

```
const int N = 1e7 + 10; // 10^7 级别
int primes[N], cnt; // primes存质数, cnt存质数个数
bool st[N]; // st[i]为true表示i是合数(被筛掉了), false表示是质数

void get_primes(int n) {
    for (int i = 2; i <= n; i++) {
        // 如果没有被标记过, 说明 i 是质数, 加入 primes 列表
        if (!st[i]) primes[cnt++] = i;
    }
}
```

```

// 核心循环：用当前的质数 primes[j] 去筛合数
for (int j = 0; primes[j] <= n / i; j++) { // primes[j] * i <= n
    // 1. 标记 primes[j] * i 为合数
    st[primes[j] * i] = true;

    // 2. 【最关键的一行】
    // 如果 i 能被 primes[j] 整除，说明 primes[j] 已经是 i 的最小质因子了
    // 那么 primes[j] 也一定是 primes[j] * i 的最小质因子
    // 我们必须立刻停止，避免重复筛
    if (i % primes[j] == 0) break;
}
}
}

int main() {
    int n = 100;
    get_primes(n);

    for(int i = 0; i < cnt; i++) {
        cout << primes[i] << " ";
    }
    return 0;
}

```

高精度

输入输出

```

#include <iostream>
#include <vector>
#include <string>
#include <algorithm> // max needs this usually
using namespace std;

// 读入处理: string -> vector<int> (倒序)
vector<int> s2v(string s) {
    vector<int> A;
    for (int i = s.size() - 1; i >= 0; i--) A.push_back(s[i] - '0');
    return A;
}

// 输出处理: 倒序打印
void print(vector<int> &A) {
    for (int i = A.size() - 1; i >= 0; i--) cout << A[i];
    cout << endl;
}

```

比大小** (减法前判断大小) **

```
bool cmp(vector<int> &A, vector<int> &B) {
    if (A.size() != B.size()) return A.size() > B.size();
    for (int i = A.size() - 1; i >= 0; i--) {
        if (A[i] != B[i]) return A[i] > B[i];
    }
    return true; // A == B
}
```

加法

```
vector<int> add(vector<int> &A, vector<int> &B) {
    vector<int> C;
    int t = 0; // 进位
    for (int i = 0; i < A.size() || i < B.size(); i++) {
        if (i < A.size()) t += A[i];
        if (i < B.size()) t += B[i];
        C.push_back(t % 10);
        t /= 10;
    }
    if (t) C.push_back(t);
    return C;
}
```

减法 必须判断大小

```
vector<int> sub(vector<int> &A, vector<int> &B) {
    vector<int> C;
    int t = 0; // 借位
    for (int i = 0; i < A.size(); i++) {
        t = A[i] - t;
        if (i < B.size()) t -= B[i];
        C.push_back((t + 10) % 10); // 借位处理技巧
        if (t < 0) t = 1; else t = 0;
    }
    // 去除前导0 (即vector末尾的0), 主要为了防止结果像 007 这种情况
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}
```

乘法

```

vector<int> mul(vector<int> &A, vector<int> &B) {
    // 结果长度最多为 A+B
    vector<int> C(A.size() + B.size(), 0);

    for (int i = 0; i < A.size(); i++) {
        for (int j = 0; j < B.size(); j++) {
            C[i + j] += A[i] * B[j]; // 先累加, 最后统一处理进位
        }
    }

    int t = 0;
    for (int i = 0; i < C.size(); i++) {
        C[i] += t;
        t = C[i] / 10;
        C[i] %= 10;
    }

    // 去除前导0
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}

```

除法

```

#include <algorithm> // 需要用到 reverse

// A: 被除数 (vector), b: 除数 (int), r: 余数 (引用传回)
vector<int> div(vector<int> &A, int b, int &r) {
    vector<int> C;
    r = 0;
    // 从高位 (vector末尾) 开始除
    for (int i = A.size() - 1; i >= 0; i--) {
        r = r * 10 + A[i]; // 上一位余数移位 + 当前位
        C.push_back(r / b); // 产生商的这一位
        r %= b;             // 更新余数
    }

    // C现在的顺序是 [最高位, ..., 个位], 不符合我们系统 [个位, ..., 最高位] 的规矩
    // 所以必须翻转
    reverse(C.begin(), C.end());

    // 去除前导0
    while (C.size() > 1 && C.back() == 0) C.pop_back();

    return C;
}

```

调用方法

```
int main() {
    string s1, s2;
    cin >> s1 >> s2;

    vector<int> A = s2v(s1);
    vector<int> B = s2v(s2);

    // 加法
    vector<int> C_add = add(A, B);
    print(C_add);

    // 减法 (处理负号逻辑)
    if (cmp(A, B)) {
        vector<int> C_sub = sub(A, B);
        print(C_sub);
    } else {
        cout << "-";
        vector<int> C_sub = sub(B, A); // 大减小
        print(C_sub);
    }

    // 乘法
    vector<int> C_mul = mul(A, B);
    print(C_mul);

    //=====除法=====//
    string s;
    int b;
    cin >> s >> b; // 读入大整数 s 和 普通整数 b

    vector<int> A = s2v(s); // 记得用上一条回答里的 s2v 转一下
    int r = 0; // 用于接收余数

    vector<int> C = div(A, b, r);

    // 输出商
    print(C); // 用上一条回答里的 print

    // 输出余数
    cout << r << endl;

    return 0;

    return 0;
}
```

组合数学

- **排列数**: $P(n, k)$ 或 $A_n^k \frac{n!}{(n-k)!}$ 使用阶乘预处理或循环计算。
- **组合数**: $C(n, k)$ 或 $\binom{n}{k} \frac{n!}{k!(n-k)!}$ 重点: 通常需要预处理或根据 k 的大小选择算法
- **阶乘**: $n! = n \times (n-1) \times \dots \times 1$ 基础预处理元素。

O(1)组合数取模结果: 阶乘逆元

```
#include <iostream>
#include <vector>

using namespace std;

using ll = long long;

// ===== 配置区域 =====
const int MAXN = 200005; // 根据题目最大范围修改 (例如 2e5 或 1e6)
const int MOD = 1e9 + 7; // 题目要求的模数 (通常是 1e9+7 或 998244353)
// =====

ll fac[MAXN]; // 阶乘数组: fac[i] = i!
ll invFac[MAXN]; // 阶乘逆元数组: invFac[i] = 1/(i!)

// 快速幂计算 (base^exp) % mod
ll qpow(ll base, ll exp) {
    ll res = 1;
    while (exp > 0) {
        if (exp & 1) res = res * base % MOD;
        base = base * base % MOD;
        exp >>= 1;
    }
    return res;
}

// 初始化函数 - 时间复杂度 O(N)
// 必须在主函数最开始调用一次 init()
void init() {
    fac[0] = 1;
    for (int i = 1; i < MAXN; i++) {
        fac[i] = fac[i - 1] * i % MOD;
    }

    // 核心优化: 先算最大阶乘的逆元, 再倒推
    // 费马小定理: inv(a) = a^(MOD-2)
    invFac[MAXN - 1] = qpow(fac[MAXN - 1], MOD - 2);

    for (int i = MAXN - 2; i >= 0; i--) {
        invFac[i] = invFac[i + 1] * (i + 1) % MOD;
    }
}

// 组合数查询 - 时间复杂度 O(1) C_n^m, 即从 n个里选 m 个。
ll nCr(int n, int m) {
```

```

    if (m < 0 || m > n) return 0; // 非法情况返回0
    return fac[n] * invFac[m] % MOD * invFac[n - m] % MOD;
}

// 排列数查询 A(n, m) - 时间复杂度 O(1)
ll nPr(int n, int m) {
    if (m < 0 || m > n) return 0;
    return fac[n] * invFac[n - m] % MOD;
}

int main() {
    // 优化 I/O 速度 (ACM 必备)
    ios::sync_with_stdio(0);
    cin.tie(0);

    // 1. 必须先初始化!
    init();

    // 2. 模拟使用场景
    int t;
    // 假设有多组查询
    // cin >> t;
    t = 3;

    // 示例查询
    // C(5, 2) = 10
    cout << "C(5, 2) = " << nCr(5, 2) << endl;

    // C(10, 3) = 120
    cout << "C(10, 3) = " << nCr(10, 3) << endl;

    // 大数测试
    cout << "C(1000, 500) % MOD = " << nCr(1000, 500) << endl;

    return 0;
}

```

好的不能再好，泰拉大师投资课

```

#include <iostream>
#include <vector>

using namespace std;

using ll = long long;

// ===== 配置区域 =====
const int MAXN = 200005; // 根据题目最大范围修改 (例如 2e5 或 1e6)
const int MOD = 1e9 + 7; // 题目要求的模数 (通常是 1e9+7 或 998244353)
// =====

```

```
11 fac[MAXN]; // 阶乘数组: fac[i] = i!
11 invFac[MAXN]; // 阶乘逆元数组: invFac[i] = 1/(i!)

// 快速幂计算 (base^exp) % mod
11 qpow(11 base, 11 exp)
{
    11 res = 1;
    while (exp > 0)
    {
        if (exp & 1)
            res = res * base % MOD;
        base = base * base % MOD;
        exp >>= 1;
    }
    return res;
}

// 初始化函数 - 时间复杂度 O(N)
// 必须在主函数最开始调用一次 init()
void init()
{
    fac[0] = 1;
    for (int i = 1; i < MAXN; i++)
    {
        fac[i] = fac[i - 1] * i % MOD;
    }

    // 核心优化: 先算最大阶乘的逆元, 再倒推
    // 费马小定理: inv(a) = a^(MOD-2)
    invFac[MAXN - 1] = qpow(fac[MAXN - 1], MOD - 2);

    for (int i = MAXN - 2; i >= 0; i--)
    {
        invFac[i] = invFac[i + 1] * (i + 1) % MOD;
    }
}

// 组合数查询 - 时间复杂度 O(1)
11 nCr(int n, int m)
{
    if (m < 0 || m > n)
        return 0; // 非法情况返回0
    return fac[n] * invFac[m] % MOD * invFac[n - m] % MOD;
}

// 排列数查询 A(n, m) - 时间复杂度 O(1)
11 nPr(int n, int m)
{
    if (m < 0 || m > n)
        return 0;
    return fac[n] * invFac[n - m] % MOD;
}

int main()
```

```

{

    ios::sync_with_stdio(0);
    cin.tie(0);

    // 1. 必须先初始化!
    init();
    int t = 0;
    cin >> t;
    while (t--)
    {
        int n;
        ll p_in, q_in; // 用 p_in, q_in 区分输入, 避免搞混
        cin >> n >> p_in >> q_in;

        // 特判 n=0 或 n=1 的情况 (视题目定义, 防一手)
        if (n == 0) { cout << 1 << "\n"; continue; }

        // 1. 算出单次赚钱概率 P 和 亏钱概率 Q
        ll inv_q_in = qpow(q_in, MOD - 2); // 分母的逆元
        ll P = p_in * inv_q_in % MOD; // P = p/q
        ll Q = (q_in - p_in) * inv_q_in % MOD; // Q = (q-p)/q

        // 2. 准备计算
        // 总共最多进行 M = 2n - 1 轮
        int M = 2 * n - 1;

        // 我们需要计算 i 从 n 到 M 的求和
        // 初始状态: 当 i = n 时
        // P 的幂次是 P^n
        ll cur_P_pow = qpow(P, n);
        // Q 的幂次是 Q^(M-n) = Q^(n-1)
        ll cur_Q_pow = qpow(Q, n - 1);

        // 为了快速更新 Q 的幂次 (从 n-1 降到 0), 我们需要 Q 的逆元
        ll inv_Q = qpow(Q, MOD - 2);

        ll ans = 0;

        // 3. 循环累加: 计算 "在 2n-1 次中, 恰好赢 i 次" 的概率
        for (int i = n; i <= M; i++)
        {
            // 项 = C(M, i) * P^i * Q^(M-i)
            ll term = nCr(M, i) * cur_P_pow % MOD * cur_Q_pow % MOD;
            ans = (ans + term) % MOD;

            // 准备下一次循环的幂次: P 多乘一次, Q 少乘一次
            cur_P_pow = cur_P_pow * P % MOD;
            cur_Q_pow = cur_Q_pow * inv_Q % MOD;
        }

        cout << ans << '\n';
    }
}

```

```
    return 0;
}
```

字符串处理

马拉车->寻找回文串

```
// Manacher算法求最长回文子串
int manacher(string s) {
    // 1. 预处理字符串: "aba" -> "$#a#b#a#^"
    // $ 和 ^ 是哨兵字符, 防止越界, # 是分隔符
    string t = "$#";
    for (char c : s) {
        t += c;
        t += '#';
    }
    t += '^'; // 尾部哨兵

    int n = t.size();
    vector<int> p(n, 0); // p[i] 表示以 t[i] 为中心的回文半径
    int mid = 0, r = 0; // mid 是当前右边界最远的回文串中心, r 是该回文串的右边界
    int maxLen = 0;

    for (int i = 1; i < n - 1; i++) {
        // 核心逻辑: 利用对称性初始化 p[i]
        // i 关于 mid 的对称点是 2*mid - i
        if (i < r) {
            p[i] = min(p[2 * mid - i], r - i);
        } else {
            p[i] = 1;
        }

        // 中心扩展
        while (t[i - p[i]] == t[i + p[i]]) {
            p[i]++;
        }

        // 更新最右边界和中心
        if (i + p[i] > r) {
            mid = i;
            r = i + p[i];
        }

        // 记录最大长度
        // 原始回文串长度 = 新串半径 p[i] - 1
        maxLen = max(maxLen, p[i] - 1);
    }

    return maxLen;
}
```

```
int main() {
    // 开启IO加速
    ios::sync_with_stdio(false);
    cin.tie(0);

    string s;
    if (cin >> s) {
        cout << manacher(s) << endl;
    }

    return 0;
}
```

比赛标准模板库：常用库函数

__builtin 位运算

高效位运算，`bit` 库的替代品。

注意：在给 64 位整数使用时须在函数名末尾添加 `ll`。

- `__builtin_ctz(x)`：返回括号内数的二进制表示**末尾 0 的个数**。
- `__builtin_clz(x)`：返回括号内数的二进制表示**前导 0 的个数**。
- `__builtin_popcount(x)`：返回括号内数的二进制表示**1 的个数**。
- `__builtin_parity(x)`：判断括号中数的二进制表示**1 的个数的奇偶性**（偶数返回 `0`，奇数返回 `1`）。

algorithm 库

`algorithm` 库提供通用算法函数（如 `sort`、`find`、`reverse`、`max`），用于排序、搜索、遍历及容器操作，支持迭代器模式，适用于数据批量处理和竞赛编程优化。

注意：

- 库中函数基本都有 `ranges` 版本
- 基于比较的函数可传入 `cmp` 以指定比较函数。

函数	描述
<code>find(v.begin(), v.end(), val)</code>	顺序查找，其中 <code>val</code> 为需要查找的值。
<code>reverse(v.begin(), v.end())</code>	翻转序列，如数组、字符串。
<code>sort(v.begin(), v.end(), cmp)</code>	内省排序，不保持相对顺序，其中 <code>cmp</code> 为自定义的比较函数。
<code>stable_sort(v.begin(), v.end(), cmp)</code>	归并排序， 保持相对顺序 ，用法同 <code>sort</code> 。

函数	描述
<code>unique(v.begin(), v.end())</code>	将序列 相邻的重复元素 移动到序列末尾，返回指向去重后序列结尾的迭代器， 原容器大小不变 。与 <code>sort</code> 结合使用可以实现序列去重。
<code>shuffle(v.begin(), v.end(), rng)</code>	随机地打乱序列（需 <code><random></code> 和随机引擎 <code>rng</code> ）。
<code>nth_element(v.begin(), v.begin() + n, v.end(), cmp)</code>	按指定范围进行分类，即找出序列中 第 <code>n</code> 大的元素 ，使其左边均为小于它的数，右边均为大于它的数。
<code>binary_search(v.begin(), v.end(), val)</code>	二分查找。其中 <code>val</code> 为需要查找的值。
<code>merge(v1.begin(), v1.end(), v2.begin(), v2.end(), v3.begin())</code>	将 有序序列 <code>v1</code> 和 <code>v2</code> 合并到序列 <code>v3</code> 。
<code>inplace_merge(v.begin() + 1, v.begin() + m, v.begin() + r)</code>	将两个（ 已按小于运算符排序的 ） <code>[1, m)</code> 、 <code>[m, r)</code> 范围 原地合并 为一个有序序列。
<code>lower_bound(v.begin(), v.end(), val)</code>	在 有序序列 <code>v</code> 中进行二分查找，返回指向 第一个大于等于 <code>val</code> 的元素 的迭代器。如果不存在，则返回尾迭代器。
<code>upper_bound(v.begin(), v.end(), val)</code>	在 有序序列 <code>v</code> 中进行二分查找，返回指向 第一个大于 <code>val</code> 的元素 的迭代器。如果不存在，则返回尾迭代器。
<code>is_permutation(v1.begin(), v1.end(), v2.begin())</code>	判断序列 <code>v1</code> 是否为 <code>v2.begin()</code> 指向的序列的 排列 ，即排序后集合元素两两相同。
<code>next_permutation(v.begin(), v.end())</code>	将当前排列更改为 全排列中的下一个排列 。如果当前已经是最后一个（ 从大到小 ），返回 <code>false</code> 并改为第一个（ 从小到大 ）；否则返回 <code>true</code> 。
<code>prev_permutation(v.begin(), v.end())</code>	将当前排列更改为 全排列中的上一个排列 ，用法同 <code>next_permutation</code> 。
<code>includes(v1.begin(), v1.end(), v2.begin(), v2.end())</code>	判断序列 <code>v1</code> 是否 包含 序列 <code>v2</code> （不必连续）。
<code>max(a, b)</code> 或 <code>max({a, b, ...})</code>	返回两个数或初始化列表中元素的最大值。
<code>min(a, b)</code> 或 <code>min({a, b, ...})</code>	返回两个数或初始化列表中元素的最小值。
<code>max_element(v.begin(), v.end())</code>	返回序列中 最大元素 的迭代器。
<code>min_element(v.begin(), v.end())</code>	返回序列中 最小元素 的迭代器。
<code>clamp(v, l, r)</code>	如果 <code>v</code> 属于 <code>[l, r]</code> ，则返回 <code>v</code> ，否则返回 最临近的边界 。

函数	描述
<code>is_partitioned(v.begin(), v.end(), pred)</code>	判断范围是否已按给定的谓词 <code>pred</code> 划分，即所有满足 <code>pred</code> 的元素会在所有不满足的元素之前。
<code>partition(v.begin(), v.end(), pred)</code>	划分序列，使得 <code>pred</code> 返回 <code>true</code> 的元素位于 <code>false</code> 的元素之前。 不保持相对顺序。
<code>stable_partition(v.begin(), v.end(), pred)</code>	稳定划分 ，用法同 <code>partition</code> 。
<code>partition_point(v.begin(), v.end(), pred)</code>	检查序列分区范围，并找到 第一个不满足 <code>p</code> 的元素 ，如果所有满足则返回尾迭代器。
<code>transform(v.begin(), v.end(), dest, op)</code>	应用函数 <code>op</code> 到输入范围，并将结果存储到 <code>dest</code> 指向的序列。
<code>replace(v.begin(), v.end(), old_value, new_value)</code>	以 <code>new_value</code> 替换 序列中所有值为 <code>old_value</code> 的元素。

string 库

`string` 库提供字符串处理功能（如 `find`、`substr`、`stoi`），用于文本操作、转换及解析，支持动态内存管理和高效字符串算法，适用于输入处理、数据格式化及竞赛编程中的文本需求。

- `to_string(value)`：将值 `value` 转化为字符串。
- `stoi(str)`：将字符串 `str` 转化为**有符号整数**。
- `stof(str)`：将字符串 `str` 转化为**浮点数**。

cctype 库

`cctype` 库提供字符处理函数（如 `isdigit`、`isalpha`、`toupper`），用于字符分类、转换及验证，支持基于 ASCII 的高效文本操作，适用于输入解析或格式化处理。

- `isalnum(ch)`：检查字符 `ch` 是否为**字母或数字**。
- `isalpha(ch)`：检查字符 `ch` 是否为**字母**。
- `islower(ch)`：检查字符 `ch` 是否为**小写字符**。
- `isupper(ch)`：检查字符 `ch` 是否为**大写字符**。
- `isdigit(ch)`：检查字符 `ch` 是否为**数字**。
- `ispunct(ch)`：检查字符是否为**标点符**（默认： `!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~``）。

numeric 库

`numeric` 库提供数值计算函数（如 `accumulate`、`inner_product`、`partial_sum` 和 `adjacent_difference`），用于累加、内积、前缀和及差分等算法操作。

- `gcd(m, n)`：计算整数 `m` 和 `n` 的**最大公约数**。
- `lcm(m, n)`：计算整数 `m` 和 `n` 的**最小公倍数**。
- `midpoint(a, b)`：计算整数、浮点或指针 `a` 和 `b` 的**中点**。
- `lerp(a, b, t)`：如果 `t` 在 `[0, 1]` 内，则计算 `a` 和 `b` 的**线性插值**，否则为**线性外推**：`a + t * (b - a)`。

- `accumulate(v.begin(), v.end(), value)`: 计算给定值 `init` 与序列中各元素的**和**。
- `iota(v.begin(), v.end(), value)`: 用**按顺序递增的值**填充序列, 从 `value` 开始 (`++value`) 。
- `partial_sum(v.begin(), v.end(), dest)`: 计算序列**前缀和**, 赋值给 `dest` 指向的序列。
- `adjacent_difference(v.begin(), v.end(), dest)`: 计算序列**差分**, 用法同前缀和。

cmath 库

`cmath` 库提供数学计算函数 (如 `sqrt`、`pow`、`abs`、`sin`) , 涵盖数值运算、几何计算及科学问题, 支持浮点和整数类型的数学操作与常用算法需求。

注意:

- 函数一般接收**浮点型输入并输出浮点型**, 注意精度损失。
- 如需更高精度, 可在函数名后添加 `l` (如 `sqrtl`) 以指定 `long double`。

函数	描述
<code>abs(x)</code>	<code>x</code> 的 绝对值 。
<code>exp(x)</code>	<code>e</code> 的 <code>x</code> 次幂。
<code>pow(x, y)</code>	<code>x</code> 的 <code>y</code> 次幂。
<code>log(x)</code>	以 <code>e</code> 为底 <code>x</code> 的对数。
<code>log10(x)</code>	以 <code>10</code> 为底 <code>x</code> 的对数。
<code>log2(x)</code>	以 <code>2</code> 为底 <code>x</code> 的对数。
<code>sqrt(x)</code>	<code>x</code> 的 平方根 。
<code>cbrt(x)</code>	<code>x</code> 的 立方根 。
<code>hypot(x, y)</code>	计算 <code>x</code> 和 <code>y</code> 平方和的平方根 。
<code>ceil(x)</code>	向上取整 (不小于 <code>x</code> 的整数) 。
<code>floor(x)</code>	向下取整 (不大于 <code>x</code> 的整数) 。
<code>round(x)</code>	四舍五入取整 (不管正负) 。
<code>trunc(x)</code>	向零取整 (舍去小数部分) 。

bit 库

`bit` 库提供底层位操作函数 (如 `countl_zero`、`bit_ceil`、`byteswap`) , 用于位计数、掩码生成和字节序转换, 支持高效位运算及数值优化。

注意: 使用 `bit` 库需要 `C++20` 以上版本。以下函数中 `x` 为**无符号整型**。

- `has_single_bit(x)`: 检查 `x` 是否为**二的整数次幂**。
- `bit_ceil(x)`: **向上取二**的整数次幂 (不小于 `x` 的最小二的幂) 。
- `bit_floor(x)`: **向下取二**的整数次幂 (不大于 `x` 的最大二的幂) 。
- `bit_width(x)`: 若 `x` 非零, 计算 `x` 的**二进制位数**; 若 `x` 为零, 返回零。
- `rotl(x, s)`: 计算 `x` **左循环移位** `s` 位的结果。

- `rotr(x, s)`: 计算 x 右循环移位 s 位的结果。
 - `countl_zero(x)`: 从最高位起计量连续的 0 位数量。
 - `countl_one(x)`: 从最高位起计量连续的 1 位数量。
 - `countr_zero(x)`: 从最低位起计量连续的 0 位数量。
 - `countr_one(x)`: 从最低位起计量连续的 1 位数量。
 - `pop_count(x)`: 返回 x 中 1 的个数。
-

二分

- 适合答案有单调性

整数二分

```
//这个是找左边界的
// 查找区间 [l, r]
while (l < r) {
    long long mid = (l + r) >> 1; // 向下取整, 不需要 +1
    if (check(mid)) {
        r = mid; // 答案在左边 (包含 mid)
    } else {
        l = mid + 1; // 答案在右边
    }
}
// 循环结束时 l == r, 即为答案
return l;
```

浮点数二分

```
for(int i=0;i<=100;i++)
{
    double mid = (l + r) / 2;
    if (check(mid, jd))
        r = mid;
    else
        l = mid;
}
cout << fixed << setprecision(10) << l << '\n';
```

三分

- 答案有凸函数的性质

整数三分

```

// 假设求【极小值】(谷底)
// 如果求极大值, 只需把 check(m1) < check(m2) 改成 check(m1) > check(m2)
// 并且最后的暴力找最大即可
ll ternary_search(ll l, ll r) {
    // 1. 核心循环: 保证区间长度 > 2 时才三分
    // 有效避免 (0,1), (0,2) 这种死循环边界
    while (r - l > 2) {
        ll m1 = l + (r - l) / 3;
        ll m2 = r - (r - l) / 3;

        // 核心判断: 谷底在较小值那边
        if (check(m1) < check(m2)) {
            r = m2; // 谷底在 m2 左侧 (含m2)
        } else {
            l = m1; // 谷底在 m1 右侧 (含m1)
        }
    }

    // 2. 暴力扫尾: 区间长度 <= 3, 直接暴力找
    // 这一步虽然土, 但是它是金牌的保障, 绝对不出错
    ll ans = l;
    for (ll i = l + 1; i <= r; i++) {
        if (check(i) < check(ans)) {
            ans = i;
        }
    }
    return ans; // 返回极值点位置, 如果要求极值本身, 返回 check(ans)
}

```

浮点数三分

```

// 浮点数三分模板
// 传入: 搜索范围 [l, r]
// 返回: 极值点的横坐标 (如果求纵坐标, 外面再套一个 f() 即可)
double ternary_search(double l, double r) {
    // 100次循环足够保证精度, 一般题目60-100次均可
    for (int i = 0; i < 100; i++) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;

        // 注意: 这里决定了是求极大值还是极小值
        // 当前逻辑: f(m1) < f(m2) 时, 说明左边更小, 且我们想找更小的 -> 舍弃右边
        // 所以这是【求极小值】
        if (f(m1) < f(m2)) {
            r = m2;
        } else {
            l = m1;
        }
    }

    // 如果要求【极大值】, 只需改为:

```

```

        // if (f(m1) < f(m2)) l = m1; else r = m2;
    }
    return l; // 最后 l 和 r 极度接近, 返回 l 或 r 或 (l+r)/2 都可以
}

```

前缀和/差分

- 区间修改, 单点查询
- 注意区间是左闭右开还是左闭右闭
- 差分和前缀和就像导数和积分一样, 使用的时候你可以用差分做到线性增长(详情见题)

一维

```

//差分
D[l] += v;
D[r + 1] -= v;
//前缀和
// 此时 D[i] 已经变成了原数组 A[i]
for(int i = 1; i <= n; i++) {
    D[i] += D[i - 1];
}

```

二维

```

D[x1][y1] += v;
D[x1][y2 + 1] -= v;
D[x2 + 1][y1] -= v;
D[x2 + 1][y2 + 1] += v;

//前缀和还原
// 假设 D 数组初始化好, 且从 (1,1) 开始
for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= m; j++) {
        // 注意: 这里不仅是累加自己, 还要复原二维前缀和公式
        // 现在的 D[i][j] 存的是差分
        // 还原成原数组 A[i][j]:
        D[i][j] += D[i-1][j] + D[i][j-1] - D[i-1][j-1];
    }
}

```

二级差分 (实现等差数列)

```

// A (原数组)
// D1 (一阶差分)
// D2 (二阶差分)
void add_AP(int l, int r, int s, int d) {

    D2[l] += s;
    D2[l + 1] += d - s;
    D2[r + 1] -= s + (long long)(r - l + 1) * d;
    D2[r + 2] += s + (long long)(r - l) * d;
}

//还原
// 1. 计算 D1
for(int i = 1; i <= n; i++) D2[i] += D2[i-1]; // 此时 D2 变成了 D1
// 2. 计算 A
for(int i = 1; i <= n; i++) D2[i] += D2[i-1]; // 此时 D2 变成了 A

```

三阶差分

```

// 数组开大点! 涉及 r+3
long long D3[MAXN];

void add_quadratic(int l, int r, long long s, long long k, long long a) {

    D3[l] += s;
    D3[l + 1] += k - 2 * s;
    D3[l + 2] += a - k + s;

    long long len = r - l + 1;

    long long v1 = s + len * k + len * (len - 1) / 2 * a;

    long long speed_at_r1 = k + len * a;
    long long v2 = v1 + speed_at_r1;

    long long speed_at_r2 = speed_at_r1 + a;
    long long v3 = v2 + speed_at_r2;

    D3[r + 1] -= v1;
    D3[r + 2] -= (v2 - 3 * v1);
    D3[r + 3] -= (v3 - 3 * v2 + 3 * v1);
}

// “修改指令”变成“最终答案”
void build(int n) {
    // 变成二阶差分

```

```

    for (int i = 1; i <= n + 3; i++) D3[i] += D3[i - 1];
    // 变成一阶差分
    for (int i = 1; i <= n + 3; i++) D3[i] += D3[i - 1];
    // 变成原数组
    for (int i = 1; i <= n + 3; i++) D3[i] += D3[i - 1];
}

```

xor差分

- $O(1)$ 查找区间xor和/某个数字
- "区间翻转" 或者 "01状态切换"

```

D[1] ^= v;
D[r + 1] ^= v;

```

乘法差分

```

D[1] = D[1] * v % MOD;
D[r + 1] = D[r + 1] * inv(v) % MOD; // 这里需要用到你的逆元板子

```

离散化差分

```

// 离散化差分标准流程
vector<int> nums; // 存所有出现过的 l 和 r+1
vector<tuple<int, int, int>> ops; // 存操作

// 1. 读入操作, 把 l 和 r+1 扔进 nums
for(...) {
    cin >> l >> r >> v;
    ops.emplace_back(l, r, v);
    nums.push_back(l);
    nums.push_back(r + 1); // 注意是 r+1
}

// 2. 离散化去重排序
sort(all(nums));
nums.erase(unique(all(nums)), nums.end());

// 3. 查找函数
auto get_id = [&](int x) {
    return lower_bound(all(nums), x) - nums.begin() + 1;
};
//注意这是1based单位

// 4. 在离散化后的数组上做差分

```

```
vector<long long> diff(nums.size() + 5, 0);
for(auto [l, r, v] : ops) {
    diff[get_id(l)] += v;
    diff[get_id(r + 1)] -= v;
}

// 5. 还原时注意:
// 离散化后的 diff[i] 代表的是原数轴上区间 [nums[i-1], nums[i]-1] 的值
```

树上差分 (因为我不会所以以下是AI教程)

```
#include <bits/stdc++.h>
using namespace std;

// ===== 参数设置 (根据题目修改) =====
const int MAXN = 200005; // 最大节点数
const int LOGN = 20;     // log2(MAXN), 一般 20 就够用了

// ===== 全局变量 =====
vector<int> adj[MAXN]; // 邻接表存图
int fa[MAXN][LOGN];   // 倍增数组: fa[u][i] 表示 u 的第 2^i 个祖先
int depth[MAXN];      // 深度数组
long long diff[MAXN]; // 差分数组 (记得开 long long 防止爆)
long long ans[MAXN];  // 最终结果数组
int n, m;

// ===== 第一步: LCA 预处理 =====
// 功能: 计算深度、初始化倍增数组
// 调用方式: dfs_lca(root, 0, 1); (root通常是1)
void dfs_lca(int u, int p, int d) {
    depth[u] = d;
    fa[u][0] = p;
    // 初始化倍增表 (2^1, 2^2 ...)
    for (int i = 1; i < LOGN; i++) {
        fa[u][i] = fa[fa[u][i - 1]][i - 1];
    }

    for (int v : adj[u]) {
        if (v != p) {
            dfs_lca(v, u, d + 1);
        }
    }
}

// 功能: 查询 u 和 v 的最近公共祖先
int get_lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v); // 保证 u 更深

    // 1. 把 u 跳到和 v 同一层
    for (int i = LOGN - 1; i >= 0; i--) {
        if (depth[u] - (1 << i) >= depth[v]) {
            u = fa[u][i];
        }
    }
    if (u == v) return u;

    // 2. 一起跳
    for (int i = LOGN - 1; i >= 0; i--) {
        if (fa[u][i] != fa[v][i]) {
            u = fa[u][i];
            v = fa[v][i];
        }
    }
    return fa[u][0];
}
```

```

        u = fa[u][i];
    }
}

if (u == v) return u; // 如果跳到了同一个点, 说明那个点就是 LCA

// 2. u 和 v 一起向上跳, 跳到 LCA 的下面一层
for (int i = LOGN - 1; i >= 0; i--) {
    if (fa[u][i] != fa[v][i]) {
        u = fa[u][i];
        v = fa[v][i];
    }
}

return fa[u][0]; // 再往上一步就是 LCA
}

// ===== 第二步: 差分操作 (二选一) =====

// 【情况A: 点差分】 u到v路径上所有点权 +w
void update_node(int u, int v, int w) {
    int lca = get_lca(u, v);
    diff[u] += w;
    diff[v] += w;
    diff[lca] -= w;
    if (fa[lca][0] != 0) { // 防止根节点没有父节点导致越界
        diff[fa[lca][0]] -= w;
    }
}

// 【情况B: 边差分】 u到v路径上所有边权 +w
// 注意: 点 i 的值代表 "i 到 fa[i]" 这条边
void update_edge(int u, int v, int w) {
    int lca = get_lca(u, v);
    diff[u] += w;
    diff[v] += w;
    diff[lca] -= 2 * w;
}

// ===== 第三步: 统计答案 =====
// 功能: 自底向上累加, 把差分值还原成真实值
// 调用方式: dfs_calc(root, 0);
void dfs_calc(int u, int p) {
    for (int v : adj[u]) {
        if (v != p) {
            dfs_calc(v, u); // 先递归子树
            diff[u] += diff[v]; // 回溯时累加子节点的值
        }
    }
    ans[u] = diff[u]; // 此时 diff[u] 就是最终的增加量
}

// ===== 示例 Main =====

```

```

int main() {
    // 1. 读入和建图
    cin >> n >> m;
    for (int i = 1; i < n; i++) {
        int u, v; cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    // 2. 必须先跑一遍 LCA 预处理!!
    dfs_lca(1, 0, 1);

    // 3. 处理所有修改操作
    while (m--) {
        int u, v, w;
        cin >> u >> v >> w;
        // 根据题目要求选一个:
        update_node(u, v, w); // 改点
        // update_edge(u, v, w); // 改边
    }

    // 4. 还原真实值
    dfs_calc(1, 0);

    // 5. 输出答案
    // 如果是边差分, ans[1] (根节点) 应该是 0
    for (int i = 1; i <= n; i++) {
        cout << ans[i] << " ";
    }

    return 0;
}

```

现场操作手册 (一定要看!) 如果你在考场上遇到了, 按以下步骤思考:

- 判断题型: 题目是不是说“给 m 条路径 (u, v) , 让路径上的都加 k , 最后问结果”? 是就是树上差分
- 区分点/边: 题目说“修改城市的繁荣度” -> 点差分
- 题目说“修改道路的修缮次数” -> 边差分。
- 抄模板: 把上面的 adj, fa, depth, diff, dfs_lca, get_lca, dfs_calc 全部抄上去。
- 改数组大小: 看一眼题目的 N 是多少, 把 MAXN 改得比 N 大一点。
- 套公式: 点差分: u 加, v 加, lca减, lca的爸爸减。边差分: u 加, v 加, lca减双倍。
- 特别提醒: LCA 必须先初始化! 在读入 m 次询问之前, 一定要先调用 dfs_lca(1, 0, 1)。
- 最后必须跑 DFS! 所有的修改只是打标记, 最后的 dfs_calc 才是真正的计算。

双指针

对撞指针

- 线性结构：数组、字符串。
- 有序（或可排序）：题目给的是增序数组，或者你可以先 `std::sort` 一遍而不影响结果。
- 查找关系：题目让你找两个元素，满足某种计算结果（和、差、积、面积）。
- 两个数满足某种数值关系

```
// 假设 vector<int> nums 是有序的
int l = 0, r = nums.size() - 1;
while (l < r) { // 注意：如果是查找两数，通常 l < r；如果是回文串，可能需要 l <= r
    int sum = nums[l] + nums[r];
    if (sum == target) {
        // 找到了
        cout << l << " " << r << endl;
        break;
    } else if (sum < target) {
        l++; // 和太小了，左指针右移（让和变大）
    } else {
        r--; // 和太大了，右指针左移（让和变小）
    }
}
```

滑动窗口

- 适用场景：子数组/子串问题（如“满足条件的最长/最短子串”、“连续子数组的和”）。这是ACM中最常考的双指针类型。核心逻辑：两个指针 `l` 和 `r` 都从左边出发。`r` 主动向右扩展窗口，`l` 被动向右收缩窗口。

```
// 这是一个通用的滑动窗口模板
// n 是数组长度
for (int r = 0, l = 0; r < n; ++r) {
    // 1. 【进窗】：将 nums[r] 加入窗口，更新状态
    add(nums[r]);

    // 2. 【出窗】：当窗口内的状态不满足条件时 (invalid)，l 指针右移
    while (/* window needs shrink */) {
        remove(nums[l]); // 移除 nums[l] 带来的影响
        l++;             // 左边界收缩
    }

    // 3. 【更新答案】：此时窗口 [l, r] 是合法的
    ans = max(ans, r - l + 1);
}
```

快慢指针

- 适用场景：原地处理数组（去重、移动零）、链表找环、链表找中点。核心逻辑：`fast` 指针负责探路（遍历所有元素），`slow` 指针负责构建有效结果（或作为锚点）。

```

int slow = 0;
for (int fast = 0; fast < nums.size(); ++fast) {
    if (nums[fast] != val) {
        // 遇到需要的元素, 就把它扔给 slow 指针的位置
        nums[slow] = nums[fast];
        slow++;
    }
    // 如果等于 val, fast 直接跳过, slow 不动
}
return slow; // slow 即为新数组的长度

```

位运算(详情请见位运算)

数据结构

树状数组

```

struct Fenwick {
    int n;
    vector<long long> tr;

    // 初始化: 传入最大长度 n
    Fenwick(int size) : n(size), tr(size + 1, 0) {}
    //初始化2: 节省时间开销的init
    void init(int size) {
        n=size;
        tr.assign(n + 1, 0);
    }

    // 核心位运算: 获取二进制最低位的 1
    int lowbit(int x) {
        return x & -x;
    }

    // 单点修改: 在位置 x 加上 val
    void add(int x, long long val) {
        for (; x <= n; x += lowbit(x)) {
            tr[x] += val;
        }
    }

    // 查询前缀和: 查询 [1, x] 的和
    long long ask(int x) {
        long long res = 0;
        for (; x > 0; x -= lowbit(x)) {
            res += tr[x];
        }
    }
}

```

```

    }
    return res;
}

// 区间查询: 查询 [l, r] 的和
long long range_ask(int l, int r) {
    if (l > r) return 0;
    return ask(r) - ask(l - 1);
}

};

int main() {
    int n = 10;
    Fenwick bit(n); // 初始化大小

    // 1. 比如输入数组 a[1...n]
    for(int i = 1; i <= n; i++) {
        int x; cin >> x;
        bit.add(i, x); // 初始化树状数组
    }

    // 2. 单点修改: 把第 3 个数加上 5
    bit.add(3, 5);

    // 3. 区间查询: 查 [2, 5] 的和
    cout << bit.range_ask(2, 5) << endl;
}

```

常用变体思路

- 区间修改 + 单点查询: 维护一个 差分数组 的树状数组。区间 $[l, r]$ 加 $v \rightarrow \text{add}(l, v)$ 和 $\text{add}(r+1, -v)$ 。
- 单点 x 查询 $\rightarrow \text{ask}(x)$ (因为差分的前缀和就是原数组的值)。求逆序对: 权值树状数组。将数字看作下标, 出现一次就 $\text{add}(\text{num}, 1)$, 统计 $\text{ask}(\text{num}-1)$ 或 $\text{total} - \text{ask}(\text{num})$ 。

ST表

- **ST表是解决“静态”区间最值问题的工具**
- 相比线段树, ST 表最大的优势是 查询时间严格 $O(1)$, 常数极小。
- 缺点是不支持修改 (静态) 且内存占用稍大 ($O(N \log N)$)。F
- 这套版本有一个优势: 分配连续空间 (毕竟是预处理) 不需要 vector 那种 push_back 顺序加速写法

```

#include <bits/stdc++.h>
using namespace std;

// 1. 定义常量
const int MAXN = 2e5 + 5; // 根据题目数据范围修改, 例如 200005

```

```

const int K = 21;          // 2^20 > 1e6, 一般 21 就够用了

// 2. 静态数组: st[i][j] 表示从位置 i 开始, 长度为 2^j 的区间
// 为了避免 cache miss, 通常把维数小的 j 放在后面, 即 st[MAXN][21]
int st[MAXN][K];

void ST_init(int n, const vector<int> &a) {
    // a 是 1-based 的, 如果是 0-based 传入要小心
    // 初始化 j=0 (长度为1) 的情况
    for (int i = 1; i <= n; i++) {
        st[i][0] = a[i];
    }

    // 倍增处理
    // j 代表层数 (2^j), i 代表起点
    for (int j = 1; j < K; j++) {
        for (int i = 1; i + (1 << j) - 1 <= n; i++) {
            // 状态转移: 当前区间 = 左半边 op 右半边
            st[i][j] = max(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
        }
    }
}

int query(int l, int r) {
    int k = __lg(r - l + 1); // C++ 内置函数, 计算 log2
    // 左右两块覆盖, 取 max
    return max(st[l][k], st[r - (1 << k) + 1][k]);
}

int main() {
    // 假设输入 n=6
    int n = 6;
    // 注意: 为了方便 1-based, 我们把 vector 开大一点, 或者填个占位符
    vector<int> a = {0, 1, 9, 2, 8, 3, 7};

    ST_init(n, a);

    // 查询区间 [2, 4] -> 即 {9, 2, 8} 最大值是 9
    cout << query(2, 4) << endl;

    return 0;
}

```

单调栈

- **运用场景**
 - 下一个/上一个更大/更小值的位置
 - 去除重复子串
 - “矩形面积最大” (直方图)
- **精妙之处** 在栈里面存下标, 只在更新的时候“取下标”。
- **AC代码**

```
int n;
cin >> n;
vector<int> a(n);
for (int i = 0; i < n; i++)
{
    cin >> a[i];
}

stack<int> stk;
vector<int> ans(n);
for (int i = 0; i < n; i++)
{
    while (!stk.empty() && a[stk.top()] < a[i])
    {
        ans[stk.top()] = i + 1;
        stk.pop();
    }
    stk.push(i);
}

for (int i = 0; i < n; i++)
{
    cout << ans[i] << " \n"[i == n - 1];
}
```

单调队列

- **运用场景**
 - 滑动窗口维护区间最大值/最小值
 - 固定窗口的极值
 - “滑动窗口”、“区间长度固定为K”、“连续子序列的最值”
- **AC代码**

```
deque<int> dq;
vi da(n + 1);
vi xiao(n + 1);
for (int i = 1; i <= n; i++)
{
    while (!dq.empty() && dq.front() + k <= i)
    {
        dq.pop_front();
    }
    while (!dq.empty() && nums[dq.back()] > nums[i])
    {
        dq.pop_back();
    }

    dq.emplace_back(i);
    da[i] = nums[dq.front()];
}
```

```

    for (int i = k; i <= n; i++)
    {
        cout << da[i] << " \n"[i == n];
    }

```

并查集

- 关键词：“连通性”、“是否在同一个集合”、“合并集合”、“朋友圈”、“亲戚关系”。
- 典型模型：给你一堆点和边，问你某两点通不通？问你图里分成了几块？（连通分量个数）。
- 警铃：只要题目暗示“不断加边，查询两点关系”，99% 是并查集。
- 能做：极速判断连通性、合并集合（时间复杂度近乎 $O(1)$ ）。
- 不能做：拆分集合（一旦合并，很难拆开）、查询两点之间的具体路径（它只知道连通，不知道怎么走）

```

const int MAXN = 200005; // 根据题目修改
int fa[MAXN]; // father数组，存每个人的老大

// 初始化：每个人最开始的老大是自己
void init(int n) {
    for (int i = 1; i <= n; i++) {
        fa[i] = i;
    }
}

// 查：找祖宗（带路径压缩，核心！）
// 递归把沿途所有人的老大直接改成祖宗，下次查就是O(1)
int find(int x) {
    if (x == fa[x]) return x;
    return fa[x] = find(fa[x]);
}

// 并：把x和y所在的两个帮派合并
void join(int x, int y) {
    int rootX = find(x);
    int rootY = find(y);
    if (rootX != rootY) {
        fa[rootX] = rootY; // 让x的老大认Y的老大做大哥
    }
}

// 判断：x和y是不是一伙的
bool isSame(int x, int y) {
    return find(x) == find(y);
}

```

带权并查集

- **相对数值关系**：题目告诉你：A 比 B 贵 10 元，B 比 C 贵 5 元。询问：A 比 C 贵多少？（这是最简单的线性权值）
- **奇偶性/异或关系**：题目告诉你：A 和 B 奇偶性相同，B 和 C 奇偶性不同。询问：A 和 C 奇偶性相同吗？（这是模 2 的权值）循
- ****环克制关系（石头剪刀布/食物链）**：**题目告诉你：A 吃 B，B 吃 C，C 吃 A。询问：A 和 D 是什么关系？（这是模 3 的权值，经典的POJ 1182 食物链）
- **向量方向要搞清：顺着箭头是加，逆着箭头是减**

```

const int MAXN = 200005;
int fa[MAXN];
int d[MAXN]; // d[x] 表示 x 到 fa[x] 的权值关系（比如距离、差值）

void init(int n) {
    for (int i = 1; i <= n; i++) {
        fa[i] = i;
        d[i] = 0; // 自己到自己距离为0
    }
}

// 带权值的 Find: 路径压缩 + 权值更新
int find(int x) {
    if (x != fa[x]) {
        int originFa = fa[x]; // 先记下旧爸爸是谁
        fa[x] = find(fa[x]); // 递归找祖宗，并进行路径压缩

        // 核心逻辑：更新 d[x]
        // 现在的 x 到 祖宗 的距离 = x 到 旧爸爸 的距离 + 旧爸爸 到 祖宗 的距离
        d[x] += d[originFa];
    }
    return fa[x];
}

// 带权值的 Join: 已知 x 和 y 的关系是 val（即 x 到 y 的距离/差值是 val）
// 目标：把 x 的祖宗(rootX) 接到 y 的祖宗(rootY) 下面
void join(int x, int y, int val) {
    int rootX = find(x);
    int rootY = find(y);
    if (rootX != rootY) {
        fa[rootX] = rootY;

        // 核心推导（向量运算）：
        // d[rootX] + d[x] = d[y] + val
        // 所以 -> d[rootX] = d[y] + val - d[x]
        d[rootX] = d[y] + val - d[x];
    }
}

// 查 x 和 y 的关系
// 如果 isSame(x, y) 为真，那么 x 和 y 的相对距离就是 d[x] - d[y]

```

搜索

DFS

- **是否需要恢复状态**：标记的状态是属于“当前这条路径”的，还是属于“这个节点”的？
 - 需要恢复现场 (Backtracking) 场景：寻找所有解、所有路径、全排列、组合、棋盘摆放（N皇后）。核心逻辑：一个点可以在不同的路径中被重复利用。虽然在这条路径里我也许不能回头，但我在尝试下一条路径时，这个点必须是“干净”的。全排列 / 组合：数字 1 在第一位用过了，撤销后，在第二位还能用。求两点间所有路径：这条路走不通或者走完了，退回来，换个方向走，刚刚走过的点对于新路径来说是没走过的。
 - 不需要恢复现场 (No Backtracking) 场景：连通块计数 (Flood Fill)、拓扑排序、求可达性（能不能到）、图的遍历。核心逻辑：只要我来过这个点一次，这个点的任务就完成了（比如已经被染过色了，或者已经证明从这出发能到达终点了）。下次再遇到它，直接跳过，绝不重复计算。连通块：这块陆地我已经踩过了，它属于第 1 个岛，下次再从别的地方撞到它，它还是属于第 1 个岛，不用把脚印擦掉。树的遍历：树没有环，只要向下走就不可能回到父节点，天然不需要判重，更不需要回溯标记
 - 特例提示：如果是记忆化搜索 (Memoization)（比如求最长路），虽然是在找路径，但因为我们记录了 `dp[u]` 的值，所以也不需要物理上的“恢复 st”，因为 `dp` 值本身就代表了该节点的最优解，不需要重新算。

全排列

```
int path[N]; // 记录路径
bool st[N];  // 记录数字是否被使用过

void dfs(int u) {
    if (u > n) {
        // 输出 path[1...n]
        return;
    }

    for (int i = 1; i <= n; i++) {
        if (!st[i]) {
            path[u] = i;
            st[i] = true; // 标记
            dfs(u + 1);
            st[i] = false; // 回溯：恢复现场
        }
    }
}

// 调用：dfs(1);
```

组合型枚举

```
// u: 当前枚举到了第几个坑位
// start: 当前只能从哪个数开始选（避免重复）
void dfs(int u, int start) {
```

```

    if (u > k) {
        // 输出 path[1...k]
        return;
    }

    for (int i = start; i <= n; i++) {
        path[u] = i;
        dfs(u + 1, i + 1); // 下一层从 i+1 开始
        // 组合一般不需要显式 st[] 数组回溯, 因为 i+1 保证了不重复
    }
}
// 调用: dfs(1, 1);

```

连通块

```

bool st[N][N]; // 访问标记
int dx[] = {-1, 0, 1, 0}, dy[] = {0, 1, 0, -1};

void dfs(int x, int y) {
    st[x][y] = true;
    for (int i = 0; i < 4; i++) {
        int a = x + dx[i], b = y + dy[i];
        // 越界、障碍、已访问检查
        if (a < 0 || a >= n || b < 0 || b >= m) continue;
        if (g[a][b] != '0' && !st[a][b]) { // 假设 '0' 是水, 非 '0' 是陆地
            dfs(a, b);
        }
    }
}

// 主循环
int cnt = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        if (g[i][j] != '0' && !st[i][j]) {
            dfs(i, j); // 把这一个连通块全染成 true
            cnt++;
        }
}

```

```

//普通图
bool st[N];
vector<int> g[N];

void dfs(int u) {
    st[u] = true;
    for (int v : g[u]) {
        if (!st[v]) dfs(v);
    }
}

```

```
// 主循环
int cnt = 0;
for (int i = 1; i <= n; i++) {
    if (!st[i]) {
        dfs(i);
        cnt++;
    }
}
```

BFS

走迷宫

```
struct Point { int x, y; };
int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0};
int dist[N][N]; // 初始化为 -1 或 INF 表示未访问
char g[N][N]; // 地图

int bfs(Point start, Point end) {
    memset(dist, -1, sizeof(dist));
    queue<Point> q;

    q.push(start);
    dist[start.x][start.y] = 0;

    while (!q.empty()) {
        auto t = q.front(); q.pop();

        if (t.x == end.x && t.y == end.y) return dist[t.x][t.y];

        for (int i = 0; i < 4; i++) {
            int a = t.x + dx[i], b = t.y + dy[i];
            // 越界 check、障碍物 check、是否已访问 check
            if (a >= 0 && a < n && b >= 0 && b < m && g[a][b] != '#' && dist[a][b]
== -1) {
                dist[a][b] = dist[t.x][t.y] + 1;
                q.push({a, b});
            }
        }
    }
    return -1; // 无法到达
}
```

拓扑排序

- 适用于有向无环图 (DAG) 的排序或判环。

```

int in[N];           // 入度
vector<int> g[N];     // 邻接表
vector<int> ans;      // 存储结果序列

bool toposort() {
    queue<int> q;
    // 1. 入度为0的点入队
    for (int i = 1; i <= n; i++) {
        if (in[i] == 0) q.push(i);
    }

    while (!q.empty()) {
        int u = q.front(); q.pop();
        ans.push_back(u);

        for (int v : g[u]) {
            if (--in[v] == 0) { // 2. 删边, 若入度变为0则入队
                q.push(v);
            }
        }
    }
    // 如果 ans.size() == n, 说明是 DAG; 否则存在环
    return ans.size() == n;
}

```

最短路

Dijkstra 非负权最短路

- AC 代码:

```

void solve()
{
    ll n,m,s;
    cin>>n>>m>>s;
    //前面是点,后面是距离
    vector<vector<pair<int,ll>>> mp(n+1);
    rep(i,0,m-1)
    {
        int u,v,w;
        cin>>u>>v>>w;
        mp[u].push_back({v,w});
        // mp[v].push_back({u,w});这道题是有向边所以不要入两次!!
    }

    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<ll,int>>>pos ;
    vector<ll> dist(n+1,LINF);
    dist[s]=0;
    pos.emplace(0,s);
}

```

```

while(!pos.empty())
{
    auto [nowd,nowp] = pos.top();
    pos.pop();
    if(nowd>dist[nowp])continue;
    for(auto [nextp,nextd]:mp[nowp])
    {
        if(dist[nextp]>dist[nowp]+nextd)
        {
            dist[nextp]=dist[nowp]+nextd;
            pos.emplace(dist[nextp],nextp);
        }
    }
}
rep(i,1,n)
{
    cout<<dist[i]<<' ';
}
}

```

• **注意事项:**

- 注意Dijkstra算法用最小堆优化可以时间复杂度最低
- Dijkstra算法只能处理**非负权路径问题**
- 为什么不用队列? : 贪心最快, 如果是菊花图复杂度会退化到nm
- 含负权路用什么算法? 用队列

Floyd-Warshall多源最短路径 (All-Pairs Shortest Path) $O(n^3)$

```

#include <algorithm>
#include <cstring>

using namespace std;

const int N = 405; // Floyd一般用于 N <= 400 的情况
const int INF = 0x3f3f3f3f; // 无穷大, 防止加法溢出

int d[N][N]; // 邻接矩阵, d[i][j] 表示从 i 到 j 的最短距离
int n, m;    // 点数, 边数

void floyd() {
    // 核心逻辑: k 必须在最外层!
    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                // 松弛操作
                // 只有当 k 能够优化 i->j 的路径时才更新
                // 判断 d[i][k] 和 d[k][j] != INF 是为了防止无穷大相加溢出 (如果INF设
                // 得很大)
                if (d[i][k] != INF && d[k][j] != INF) {

```

```

        d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
    }
}
}
}

int main() {
    // 1. 初始化
    // 自己到自己距离为0, 到其他人默认为无穷大
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= N; j++) {
            if (i == j) d[i][j] = 0;
            else d[i][j] = INF;
        }
    }

    // 2. 读入边 (假设是带权有向图)
    // 注意: 如果是重边, 通常保留权值最小的那条
    // int u, v, w;
    // d[u][v] = min(d[u][v], w);

    // 3. 执行算法
    floyd();

    return 0;
}

```

处理负权边 (Dijkstra 的死穴) 这是最本质的区别。Dijkstra 基于贪心思想。它假设“当前找到的最短路径就是最终的最短路径”，一旦确定了一个点的最短距离，就不会再更改。如果图中存在负权边，Dijkstra 的贪心逻辑就会失效，导致答案错误。Floyd 基于动态规划。它穷举了所有的中转可能性。只要图中没有负权环 (Negative Cycle)，Floyd 就能完美处理负权边。场景判断：只要看到题目里边权可能为负数（且 N 较小），Dijkstra 直接排除，立刻选 Floyd（或 SPFA/Bellman-Ford）。多源最短路径 vs 单源最短路径 Dijkstra 是单源最短路径。算一次只能求出“起点 S 到其他所有点”的距离。如果你想求“任意两点间距离”，你需要对每个点跑一次 Dijkstra Floyd 是多源最短路径。跑一次，所有点对 (i, j) 的距离都出来了 复杂度对比（求所有点对距离）：Floyd: $O(N^3)$ N 次 Dijkstra (堆优化): $O(N \cdot M \log N)$ 如果是稀疏图 ($M \approx N$)，Dijkstra 确实更快 ($O(N^2 \log N)$)。如果是稠密图 ($M \approx N^2$)，Dijkstra 会退化成 $O(N^3 \log N)$ ，此时 Floyd 反而更快，而且常数极小。

SPFA 最坏时间复杂度: $O(N \cdot M)$

```

const int N = 100005; // 根据题目要求设定
const int INF = 0x3f3f3f3f;

// 存图: vector<pair<邻居, 权值>>
vector<pair<int, int>> adj[N];
int dist[N]; // 存储起点到各点的最短距离
bool in_queue[N]; // 标记是否在队列中 (避免重复入队)
int cnt[N]; // 记录经过的边数, 用于判断负环

```

```

int n, m;

// 返回 true 表示存在负环, false 表示正常求出了最短路
bool spfa(int s) {
    // 1. 初始化
    memset(dist, 0x3f, sizeof(dist));
    memset(in_queue, 0, sizeof(in_queue));
    memset(cnt, 0, sizeof(cnt));

    dist[s] = 0;
    queue<int> q;
    q.push(s);
    in_queue[s] = true;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        in_queue[u] = false; // 出队后标记为不在队列中

        // 扫描所有出边
        for (auto &edge : adj[u]) {
            int v = edge.first;
            int w = edge.second;

            // 松弛操作: 如果通过 u 到 v 更近
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                cnt[v] = cnt[u] + 1; // 记录路径长度

                // 判断负环: 如果一条路径经过了 >= n 个点, 说明有环
                if (cnt[v] >= n) return true;

                // 如果 v 不在队列中, 则入队
                // (这是 SPFA 比 Bellman-Ford 快的原因: 只处理更新过的点)
                if (!in_queue[v]) {
                    q.push(v);
                    in_queue[v] = true;
                }
            }
        }
    }

    return false; // 没有负环
}

int main() {
    // 读入边...
    // spfa(start_node);
    return 0;
}

```

背包DP

01背包

```
// 必须倒序枚举 j, 保证每个物品只被选中一次
for (int i = 1; i <= n; ++i) {
    for (int j = V; j >= w[i]; --j) {
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
}
```

完全背包

```
// 必须正序枚举 j, 利用当前状态更新后续状态 (允许重复选)
for (int i = 1; i <= n; ++i) {
    for (int j = w[i]; j <= V; ++j) {
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
}
```

多重背包

- 每种物品有 $s[i]$ 个。
- 思路：将 $s[i]$ 个物品拆分成 $1, 2, 4, \dots, 2^k, \text{remainder}$ 若干个“新物品”，然后跑 0/1 背包。时间复杂度从 $O(V \cdot \sum s[i])$ 降为 $O(V \cdot \sum \log s[i])$ 。

```
// 预处理：拆分物品
struct Item { int w, v; };
vector<Item> items;

for (int i = 1; i <= n; ++i) {
    int count = s[i];
    for (int k = 1; count >= k; k *= 2) {
        items.push_back({w[i] * k, v[i] * k});
        count -= k; // 注意这里填k
    }
    if (count > 0) {
        items.push_back({w[i] * count, v[i] * count});
    }
}

// 跑 0/1 背包
for (auto& item : items) {
    for (int j = V; j >= item.w; --j) {
        dp[j] = max(dp[j], dp[j - item.w] + item.v);
    }
}
```

- 二进制优化/拆分

- **解说:** $k *= 2$: 这是在生成 $1, 2, 4, 8 \dots$ 的序列。 $count -= k$: 保证我们拆分出的总数之和刚好等于原有的 $s[i]$ 。最后的 $if (count > 0)$: 这是一个“补丁”。因为 2^n 的累加不一定刚好等于 $s[i]$ 。比如 $s = 10$, 拆出 $1, 2, 4$ 后剩下 3 , 这个 3 必须单独打包, 才能凑出 $0 \sim 10$ 所有的数。

```
for (int k = 1; count >= k; k *= 2) {
    items.push_back({w[i] * k, v[i] * k}); // 把 k 个物品打包成一个整体
    count -= k; // 减去已经打包的数量
}
if (count > 0) {
    items.push_back({w[i] * count, v[i] * count}); // 处理最后剩下的不够 2 的幂次的部分
}
```

分组背包

- 物品被分为 K 组, 每组中至多只能选 1 件。

```
// 外层枚举组, 中层枚举容量 (倒序), 内层枚举组内物品
for (int k = 1; k <= groups; ++k) {
    for (int j = V; j >= 0; --j) {
        for (auto& item : group[k]) { // item 包含 {w, v}
            if (j >= item.w) {
                dp[j] = max(dp[j], dp[j - item.w] + item.v);
            }
        }
    }
}
```

二维费用背包

```
// 两个维度都倒序
for (int i = 1; i <= n; ++i) {
    for (int j = V; j >= w[i]; --j) {
        for (int k = M; k >= m[i]; --k) {
            dp[j][k] = max(dp[j][k], dp[j - w[i]][k - m[i]] + v[i]);
        }
    }
}
```

线性DP

LIS

- **题目：**给个序列nums，求他的最长上升子序列

- $O(n^2)$ 做法:

$$dp[i] = \max(dp[j]) + 1 \quad \text{其中 } 0 \leq j < i, a[i] > a[j]$$

- $O(n \log n)$ 做法:

- **注意事项**

- 代码体现:
 - 严格 (<): lower_bound(d.begin(), d.end(), x)
 - 不降 (<=): upper_bound(d.begin(), d.end(), x)
- d 数组的真实含义：如果你需要输出具体的子序列内容，不能直接打印 d，需要记录 pre 数组回溯。

```
int LIS_nlogn(vector<int>& a) {
    if (a.empty()) return 0;

    // low[i] 存的是长度为 i+1 的子序列的最小结尾
    // 注意：为了方便，low 的下标从 0 开始，所以 low[len-1] 是长度为 len 的结尾
    vector<int> low;

    for (int x : a) {
        // Case 1: 如果 low 为空，或者 x 比 low 最后一个元素大
        if (low.empty() || x > low.back()) {
            low.push_back(x);
        }
        // Case 2: 否则，在 low 里找到第一个 >= x 的位置，替换它
        else {
            // lower_bound 返回迭代器，指向第一个 >= x 的元素
            auto it = lower_bound(low.begin(), low.end(), x);
            *it = x; // 贪心：让该长度的结尾变得更小
        }
    }

    return low.size();
}
```

- **优化思考路径**

- 这是一种优化贪心，将dp对象转换成能达到的长度，每一位都代表着如果要达到这个长度我最小要有多大，精辟在更新。相当于转换了dp的两个对象：
- ==最长能有多长->如果要达到这个长度需要最小什么条件==

- ==应用==:

- **Dilworth定理：**最少的下降序列个数就等于整个序列最长上升子序列的长度
- LCS解法->将一个数组的每一个数字跟他的位置映射在一起，相当于开了个桶

- **注意：**求下降序列需要 `auto it= upper_bound(all(hei),hsh,greater<int>());`。普通的公式只能算：排列好后的不下降数列

杂

字符串哈希

- 下标建议从 1 开始逻辑（模板内部已处理，你传入 0-based 字符串即可）。
- 两个字符串子串相等的充要条件是：它们的 `get_hash()` 返回的 pair 完全相等。

```
#include <chrono>
#include <random>

// 双哈希核心结构体
struct StringHash {
    // 定义两组模数
    static const long long MOD1 = 1e9 + 7;
    static const long long MOD2 = 1e9 + 9;

    // 存储哈希前缀和
    vector<long long> h1, h2;
    // 存储 Base 的幂次
    vector<long long> p1, p2;

    // 构造函数：传入字符串 s
    StringHash(const string &s) {
        int n = s.size();
        h1.resize(n + 1); h2.resize(n + 1);
        p1.resize(n + 1); p2.resize(n + 1);

        // 随机生成 Base (131 ~ 13331 之间)，防止被卡
        mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
        long long BASE1 = uniform_int_distribution<long long>(131, 13331)(rng);
        long long BASE2 = uniform_int_distribution<long long>(131, 13331)(rng);

        // 预处理
        p1[0] = 1; p2[0] = 1;
        h1[0] = 0; h2[0] = 0;

        for (int i = 0; i < n; i++) {
            p1[i + 1] = (p1[i] * BASE1) % MOD1;
            p2[i + 1] = (p2[i] * BASE2) % MOD2;

            // s[i] 也就是第 i+1 个字符
            h1[i + 1] = (h1[i] * BASE1 + s[i]) % MOD1;
            h2[i + 1] = (h2[i] * BASE2 + s[i]) % MOD2;
        }
    }

    // 查询子串 s[l...r] 的哈希值（闭区间，1-based）
    // 例如：查询 s 的前 3 个字符，传入 get(1, 3)
```

```
pair<long long, long long> get(int l, int r) {
    long long res1 = (h1[r] - h1[l - 1] * p1[r - l + 1] % MOD1 + MOD1) % MOD1;
    long long res2 = (h2[r] - h2[l - 1] * p2[r - l + 1] % MOD2 + MOD2) % MOD2;
    return {res1, res2};
}

};

int main() {
    string s = "ababcbab";
    // 1. 初始化
    StringHash sh(s);

    // 2. 查询 s[1...2] ("ab") 和 s[3...4] ("ab")
    // 注意: 这里的下标是 1-based, 对应字符串原本的 index 0 和 1
    if (sh.get(1, 2) == sh.get(3, 4)) {
        cout << "Same!" << endl; // 输出 Same!
    } else {
        cout << "Different!" << endl;
    }

    return 0;
}
```